

Kate Leemann
ITAI 1378
Professor McManus
June 24, 2026

L04 Chihuahua or Muffin

Workshop Summary and Results

The main objective of this workshop was to make the basic neural-network training process concrete by building an image classifier in PyTorch. The task was intentionally simple: decide whether an image showed a Chihuahua or a muffin. I cloned the GitHub repository, inspected the training and validation folders, converted the images into tensors, trained a model, and then reviewed its predictions. I chose an image size of 64 by 64 pixels and a batch size of eight. These values reduced the amount of computation while preserving enough visual information for the classifier to learn useful patterns.

The model was a multi-layer perceptron rather than a convolutional neural network. Each RGB image was flattened into 12,288 individual values and passed through layers containing 128, 64, 32, and 2 neurons. ReLU activation functions introduced nonlinearity, and the final output represented the two possible classes. The model used cross-entropy loss, stochastic gradient descent, and a learning rate of 0.1. Training for three epochs improved training accuracy from 45.83% to 80.00%, while validation accuracy increased from 46.67% to 86.67%. With 30 validation images, that final score means the model classified 26 images correctly and missed four.

The prediction display was more informative than the accuracy number by itself. Some images received a clear prediction, including several muffins with probabilities near 80% to 90%. Other images produced nearly even probabilities, such as 49% Chihuahua and 51% muffin. The model also confused a few close-up Chihuahua faces with muffins and classified two muffin images as slightly more likely to be Chihuahuas. This showed that a correct class label does not always mean the model is confident, and a high overall accuracy can hide uncertain individual decisions.

Key Concepts and Challenges

The most important concept from the unit was the difference between classic machine learning and neural networks. In a classic computer-vision pipeline, a person chooses features such as edges, textures, or shapes before training the classifier. A neural network instead learns useful internal representations by adjusting its weights during training. The workshop made the learning loop easier to understand: the forward pass creates a prediction, the loss function measures the error, backpropagation traces that error through the network, and the optimizer updates the weights. Repeating this process gradually reduces the loss and improves prediction accuracy. This is the same general structure used in much larger deep-learning systems (LeCun et al.).

The main practical challenge was identifying what belonged in the intentionally blank sections of the notebook. The placeholders were not missing theory; they were asking me to connect previously defined variables to later steps. For example, the training folder needed the training transformation pipeline, the validation folder needed the validation pipeline, and the model variable needed an instance of the network class. Once I followed the data flow from image folder to transform, data loader, model, loss, and optimizer, the code became much

Kate Leemann
ITAI 1378
Professor McManus
June 24, 2026

easier to interpret. This was useful because it required understanding what each variable represented rather than only copying complete code.

The workshop also showed a major limitation of a basic multi-layer perceptron for images. Flattening the image removes the built-in relationship between neighboring pixels. The model can still learn patterns, but it has no natural understanding that adjacent pixels form an edge, texture, eye, or muffin top. Convolutional neural networks address this problem with local receptive fields and shared filters. CNNs preserve spatial structure and use far fewer parameters than a fully connected network. Transfer learning would also be a logical next step because this dataset contains only 120 training images. A pre-trained CNN could reuse visual features learned from a much larger dataset instead of learning every feature from scratch. This notebook did not implement a CNN or transfer learning, but it clearly demonstrated why those methods are useful.

Personal Reflection

One insight I gained is that model evaluation should include more than a final accuracy score. The validation set was small, so each image changed the accuracy by more than three percentage points. An 86.67% result sounds precise, but it represents only four mistakes out of 30 images. Looking at the class probabilities exposed uncertainty that the summary metric did not show. In a real application, I would want a confusion matrix, a larger independent test set, and a way to flag low-confidence predictions for human review.

The exercise also made the effects of preprocessing and hyperparameters more tangible. Image size, normalization, batch size, learning rate, network depth, and number of epochs can all change the result. This reminded me of laboratory work, where a final measurement depends on many earlier protocol choices. A model can run successfully and still produce weak or misleading results if the inputs, controls, or evaluation design are poor. PyTorch automates gradient calculation and weight updates, but the person building the model is still responsible for selecting the data, architecture, and evaluation method (Paszke et al., 2019).

Image classification has many practical applications, including medical imaging, microscopy, manufacturing inspection, agriculture, and biological quality control. In a laboratory setting, similar methods could help screen microscope images, identify colony morphology, sort image-based assay results, or flag unusual samples for review. I would treat these systems as decision-support tools rather than replacements for expert judgment, especially in medical, forensic, or other high-consequence settings.

Overall, this workshop made neural networks feel less abstract. The Chihuahua-versus-muffin example was simple enough to follow, but difficult enough to reveal the limits of the model. The most useful part was seeing the entire workflow in one notebook and connecting the code to the concepts from the lecture. I now have a clearer understanding of how data moves through a network, how learning occurs through repeated weight updates, and why CNNs and transfer learning are better suited to many real image-classification problems. It also reinforced my interest in applying machine learning to biological and bioinformatics problems, where understanding uncertainty and validating results are just as important as achieving a high accuracy score.

Kate Leemann
ITAI 1378
Professor McManus
June 24, 2026

References

LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521, 436-444.
<https://doi.org/10.1038/nature14539>

Paszke, A., Gross, S., Massa, F., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32.

Patitimoner. (n.d.). workshop-chihuahua-vs-muffin [Jupyter notebook]. GitHub.
<https://github.com/patitimoner/workshop-chihuahua-vs-muffin>